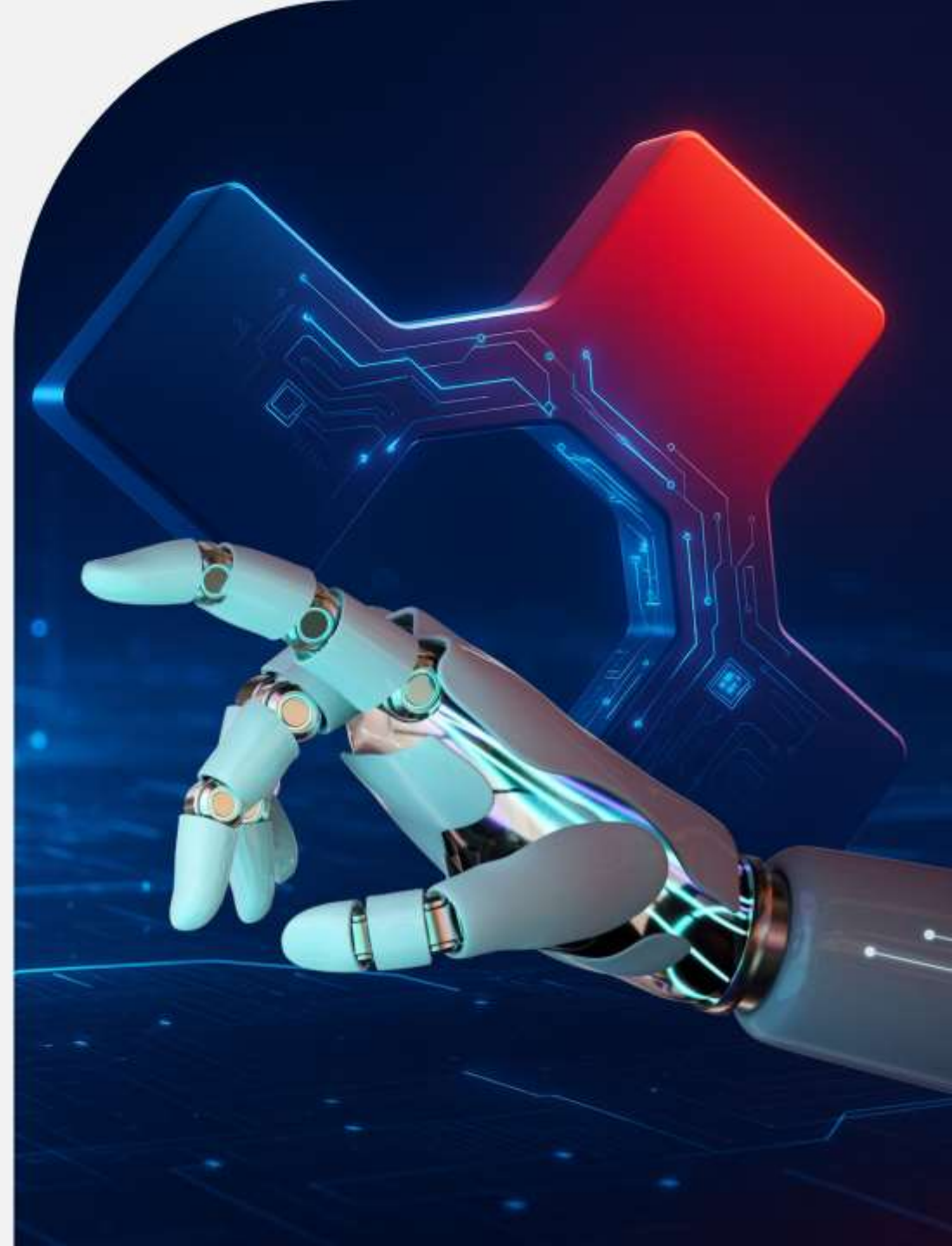
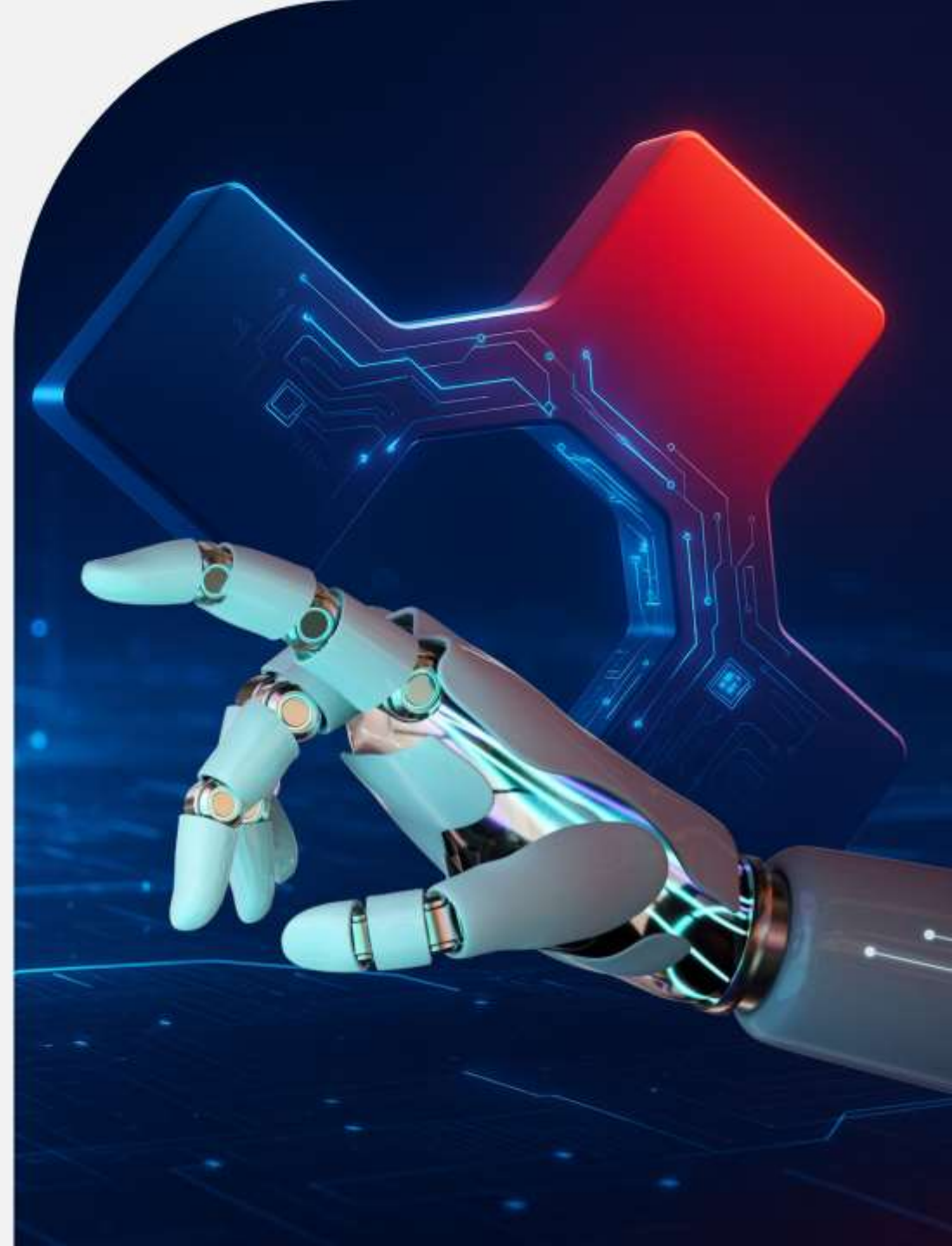


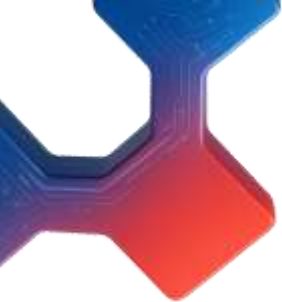


Núcleo de Capacitação em Inteligência Artificial



# Aula 17



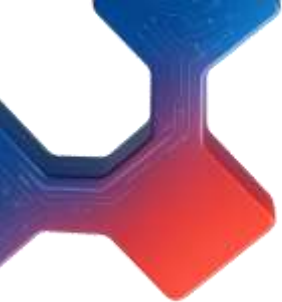


# 1. Por Dentro dos Classificadores SVM Lineares

- Um classificador SVM linear prediz a classe de uma nova instância  $\mathbf{x}$  primeiro computando a função de decisão:

$$\boldsymbol{\theta}^T \mathbf{x} = \boldsymbol{\theta}_0 \mathbf{x}_0 + \dots + \boldsymbol{\theta}_n \mathbf{x}_n,$$

onde  $\mathbf{x}_0$  é a característica de viés (sempre igual a **1**). Se o resultado for positivo, a classe prevista  $\hat{\mathbf{y}}$  é a classe positiva (**1**); caso contrário, é a classe negativa (**0**). Isso funciona exatamente como na regressão logística.



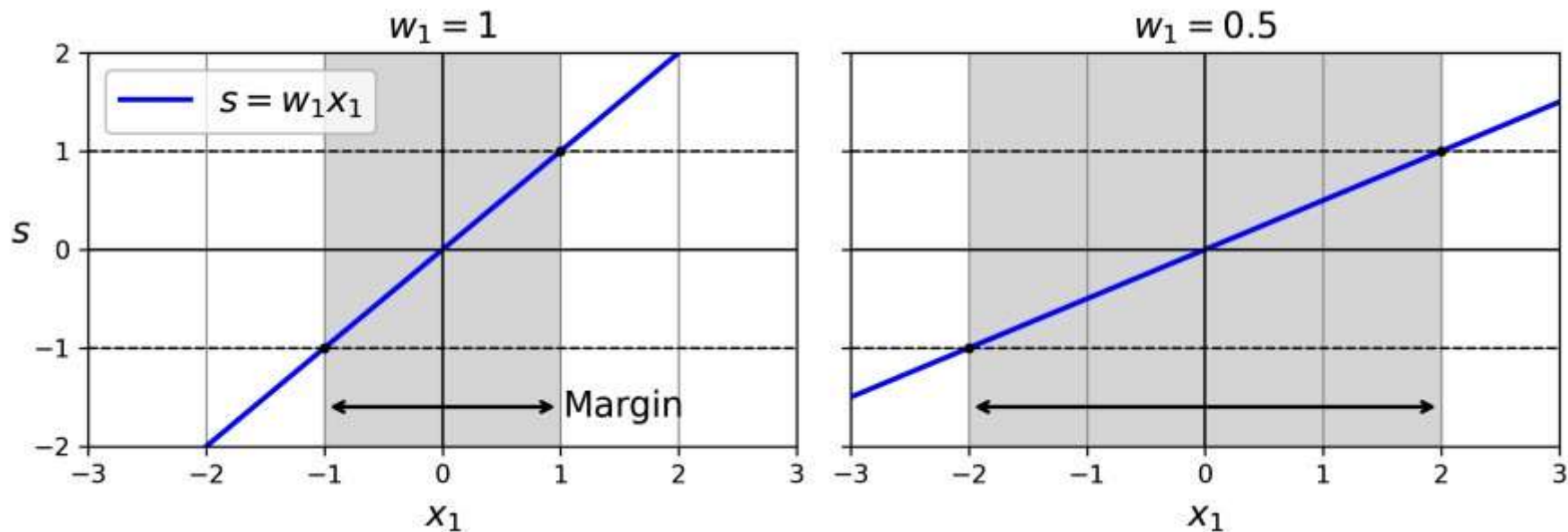
# 1. Por Dentro dos Classificadores SVM Lineares

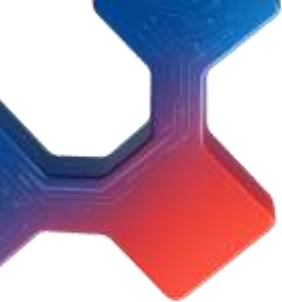
- Até agora, usamos a convenção de colocar todos os parâmetros do modelo em um único vetor  $\boldsymbol{\theta}$ , incluindo o termo de viés  $\boldsymbol{\theta}_0$  e os pesos das características de entrada  $\boldsymbol{\theta}_1$  a  $\boldsymbol{\theta}_n$ . Isso exigiu adicionar uma entrada de viés  $\mathbf{x}_0 = \mathbf{1}$  a todas as instâncias. Outra convenção muito comum é separar o termo de viés  $\mathbf{b}$  (igual a  $\boldsymbol{\theta}_0$ ) e o vetor de pesos das características  $\mathbf{w}$  (contendo  $\boldsymbol{\theta}_1$  a  $\boldsymbol{\theta}_n$ ). Nesse caso, não é necessário adicionar uma característica de viés aos vetores de entrada, e a função de decisão do SVM linear é igual a

$$\mathbf{w}^T \mathbf{x} + \mathbf{b} = \mathbf{w}_1 \mathbf{x}_1 + \cdots + \mathbf{w}_n \mathbf{x}_n + \mathbf{b}.$$

# 1. Por Dentro dos Classificadores SVM Lineares

- Então, fazer previsões com um classificador SVM linear é bastante direto. **E quanto ao treinamento?** Isso requer encontrar o vetor de pesos  $\mathbf{w}$  e o termo de viés  $\mathbf{b}$  que tornam a “rua” (ou margem) o mais larga possível, ao mesmo tempo que limitam o número de violações da margem. Vamos começar com a largura da rua: para torná-la maior, precisamos tornar  $\mathbf{w}$  menor. Isso pode ser mais fácil de visualizar em 2D, como mostrado na Figura a seguir.

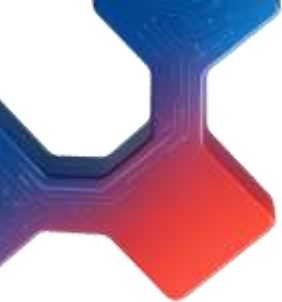




# 1. Por Dentro dos Classificadores SVM Lineares

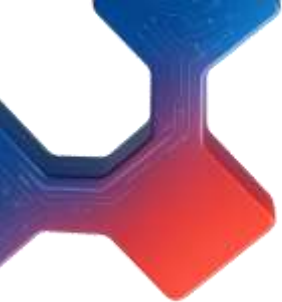
- Vamos definir as bordas da rua como os pontos onde a função de decisão é igual a  $-1$  ou  $+1$ . No gráfico à esquerda, o peso  $\mathbf{w}_1$  é 1, então os pontos nos quais  $\mathbf{w}_1 \mathbf{x}_1 = -1$  ou  $+1$  são  $\mathbf{x}_1 = -1$  e  $+1$ ; portanto, o tamanho da margem é 2. No gráfico à direita, o peso é 0,5, então os pontos nos quais  $\mathbf{w}_1 \mathbf{x}_1 = -1$  ou  $+1$  são  $\mathbf{x}_1 = -2$  e  $+2$ ; o tamanho da margem é 4. Portanto, precisamos manter  $\mathbf{w}$  o menor possível. Note que o termo de viés  $\mathbf{b}$  não influencia o tamanho da margem: ajustá-lo apenas desloca a margem, sem afetar sua largura.





# 1. Por Dentro dos Classificadores SVM Lineares

- Também queremos evitar violações da margem, então precisamos que a função de decisão seja maior que 1 para todas as instâncias de treinamento positivas e menor que  $-1$  para as negativas. Definindo  $\mathbf{t}^{(i)} = -1$  para instâncias negativas ( $\mathbf{y}^{(i)} = 0$ ) e  $\mathbf{t}^{(i)} = 1$  para instâncias positivas ( $\mathbf{y}^{(i)} = 1$ ), podemos escrever essa restrição como  $\mathbf{t}^{(i)} (\mathbf{w}^T \mathbf{x}^{(i)} + \mathbf{b}) \geq 1$  para todas as instâncias.

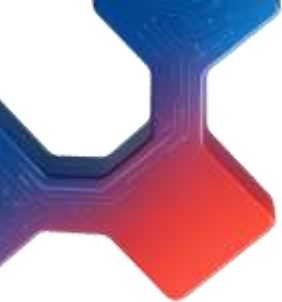


# 1. Por Dentro dos Classificadores SVM Lineares

- Podemos, portanto, expressar o objetivo do classificador **SVM linear de margem rígida** como o problema de otimização com restrições mostrado no problema a seguir (Objetivo do classificador SVM linear de margem rígida).

$$\begin{aligned} & \underset{\mathbf{w}, b}{\text{minimize}} && \frac{1}{2} \mathbf{w}^\top \mathbf{w} \\ & \text{subject to} && t^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1 \quad \text{for } i = 1, 2, \dots, m \end{aligned}$$





# 1. Por Dentro dos Classificadores SVM Lineares

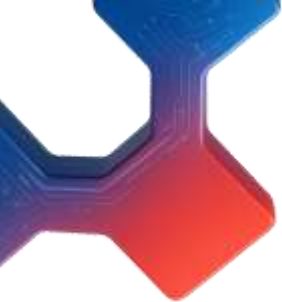
- Estamos minimizando  $\frac{1}{2} \mathbf{w}^T \mathbf{w}$ , que é igual a  $\frac{1}{2} \|\mathbf{w}\|^2$ , em vez de minimizar  $\|\mathbf{w}\|$  (a norma de  $\mathbf{w}$ ). De fato,  $\frac{1}{2} \|\mathbf{w}\|^2$  possui uma derivada simples e conveniente (que é apenas  $\mathbf{w}$ ), enquanto  $\|\mathbf{w}\|$  não é diferenciável em  $\mathbf{w} = \mathbf{0}$ . **Algoritmos de otimização geralmente funcionam muito melhor com funções diferenciáveis.**
- $a_2^2$

$$\begin{aligned} & \underset{\mathbf{w}, b}{\text{minimize}} && \frac{1}{2} \mathbf{w}^T \mathbf{w} \\ & \text{subject to} && t^{(i)} (\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 \quad \text{for } i = 1, 2, \dots, m \end{aligned}$$

# 1. Por Dentro dos Classificadores SVM Lineares

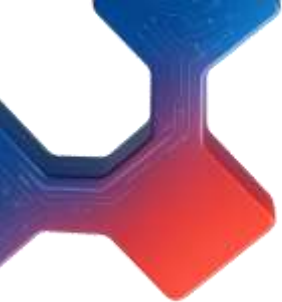
- Para obter o objetivo do SVM de margem suave, precisamos introduzir uma variável de folga  $\zeta^{(i)} \geq 0$  para cada instância, onde  $\zeta^{(i)}$  mede quanto a  $i$ -ésima instância é permitida violar a margem. Agora temos dois objetivos conflitantes: manter as variáveis de folga o menor possível para reduzir as violações da margem e manter  $\frac{1}{2} \mathbf{w}^T \mathbf{w}$  o menor possível para aumentar a margem. É nesse ponto que entra o hiperparâmetro  $C$ : ele nos permite definir o equilíbrio entre esses dois objetivos. Isso nos leva ao problema de otimização com restrições apresentado na Equação abaixo (**Objetivo do classificador SVM linear de margem suave**) .

$$\begin{aligned} & \underset{\mathbf{w}, b, \zeta}{\text{minimize}} && \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^m \zeta^{(i)} \\ & \text{subject to} && t^{(i)} (\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \zeta^{(i)} \quad \text{and} \quad \zeta^{(i)} \geq 0 \quad \text{for } i = 1, 2, \dots, m \end{aligned}$$



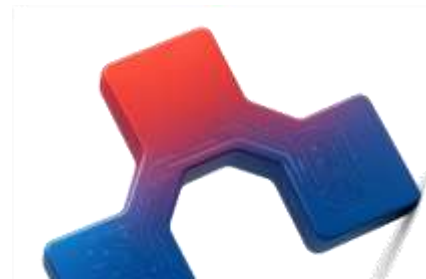
# 1. Por Dentro dos Classificadores SVM Lineares

- The hard margin and soft margin problems are both convex quadratic optimization problems with linear constraints. Such problems are known as quadratic programming (QP) problems. Many off-the-shelf solvers are available to solve QP problems by using a variety of techniques that are outside the scope of this book.
- Using a QP solver is one way to train an SVM. Another is to use gradient descent to minimize the hinge loss or the squared hinge loss (see Figure 5-13). Given an instance  $x$  of the positive class (i.e., with  $t = 1$ ), the loss is 0 if the output  $s$  of the decision function ( $s = w^T x + b$ ) is greater than or equal to 1. This happens when the instance is off the street and on the positive side

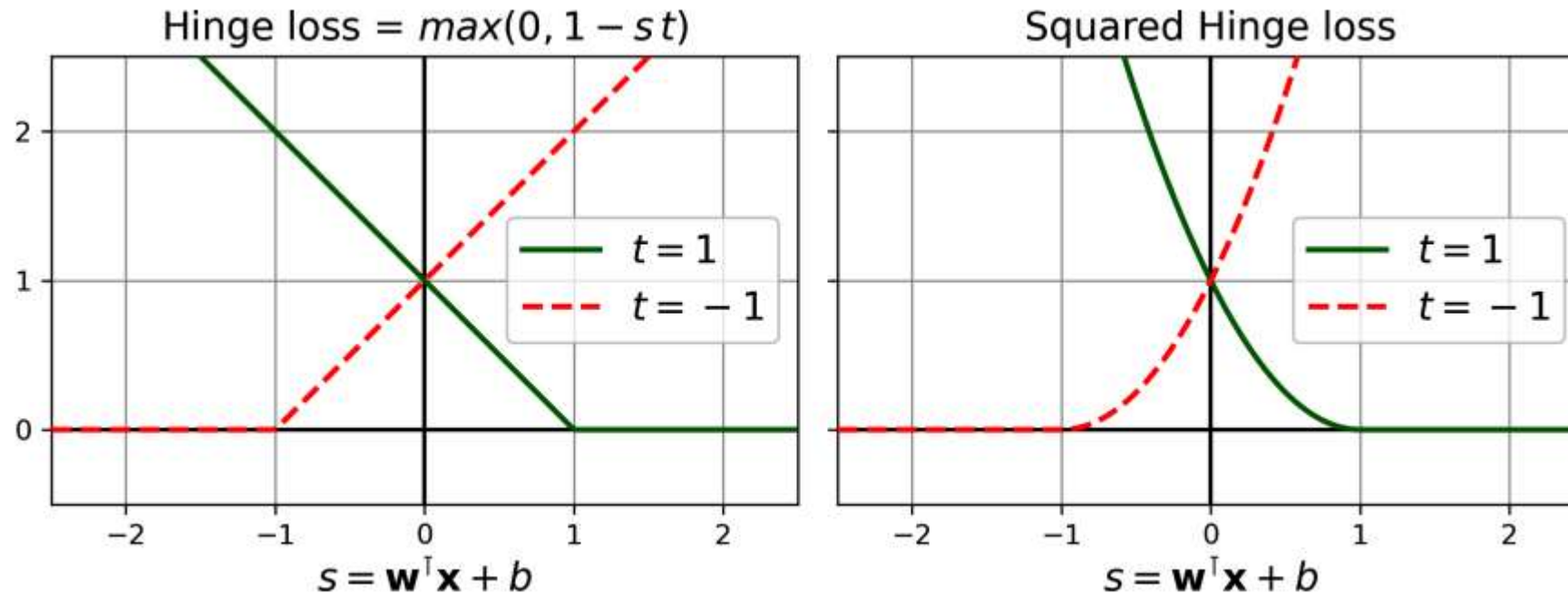


# 1. Por Dentro dos Classificadores SVM Lineares

- Given an instance of the negative class (i.e., with  $t = -1$ ), the loss is 0 if  $s \leq -1$ . This happens when the instance is off the street and on the negative side. The further away an instance is from the correct side of the margin, the higher the loss: it grows linearly for the hinge loss, and quadratically for the squared hinge loss.
- This makes the squared hinge loss more sensitive to outliers. However, if the dataset is clean, it tends to converge faster. By default, LinearSVC uses the squared hinge loss, while SGDClassifier uses the hinge loss. Both classes let you choose the loss by setting the loss hyperparameter to "hinge" or "squared\_hinge". The SVC class's optimization algorithm finds a similar solution as minimizing the hinge loss.



# 1. Por Dentro dos Classificadores SVM Lineares



Next, we'll look at yet another way to train a linear SVM classifier: solving the dual problem !!

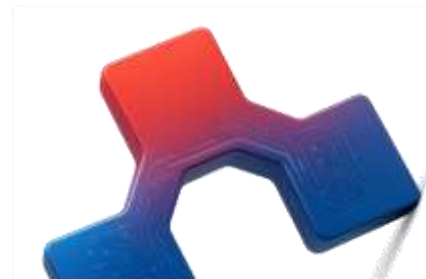


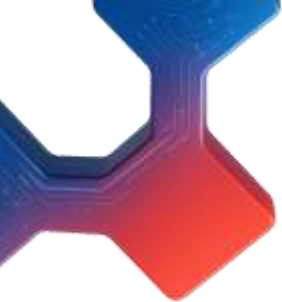
## 2. O Problema Dual

- Dado um problema de otimização com restrições, conhecido como **problema primal**, é possível expressar um problema diferente, mas intimamente relacionado, chamado de **problema dual**. A solução do **problema dual** normalmente fornece um limite inferior para a solução do problema primal, mas, sob certas condições, pode ter a mesma solução que o **problema primal**. Felizmente, o **problema do SVM atende a essas condições**, de modo que você pode optar por resolver o problema **primal** ou o **dual**; ambos terão a mesma solução.

### 1. Forma da dualidade do Objetivo da SVM linear

$$\begin{aligned} \underset{\alpha}{\text{minimize}} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha^{(i)} \alpha^{(j)} t^{(i)} t^{(j)} \mathbf{x}^{(i)\top} \mathbf{x}^{(j)} - \sum_{i=1}^m \alpha^{(i)} \\ \text{subject to} \quad & \alpha^{(i)} \geq 0 \text{ for all } i = 1, 2, \dots, m \text{ and } \sum_{i=1}^m \alpha^{(i)} t^{(i)} = 0 \end{aligned}$$





## 2. O Problema Dual

- Uma vez encontrado o vetor  $\alpha$  que minimiza essa equação (usando um solver de Programação Quadrática – QP), utilize a equação acima para calcular  $\mathbf{w}$  e  $\mathbf{b}$ , que minimizam o problema primal. Nessa equação,  $n_s$  representa o número de vetores de suporte.

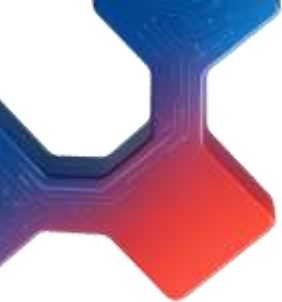
### 2. Da solução de dualidade à solução primal

$$\widehat{\mathbf{w}} = \sum_{i=1}^m \widehat{\alpha}^{(i)} t^{(i)} \mathbf{x}^{(i)}$$

$$\widehat{b} = \frac{1}{n_s} \sum_{\substack{i=1 \\ \widehat{\alpha}^{(i)} > 0}}^m \left( t^{(i)} - \widehat{\mathbf{w}}^T \mathbf{x}^{(i)} \right)$$

- O problema da **dualidade** é mais rápido de resolver do que o **primal**, quando o número de instâncias de treinamento é menor que o número de características.





## 4. SVMs kernalizados

- Suponha que você queira aplicar uma transformação polinomial de segundo grau a um conjunto de treinamento bidimensional (como o conjunto de treinamento “moons”) e, em seguida, treinar um classificador SVM linear no conjunto transformado..

$$\varphi(\mathbf{x}) = \varphi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ \sqrt{2} x_1 x_2 \\ x_2^2 \end{pmatrix}$$

## 4. SVMs kernalizados

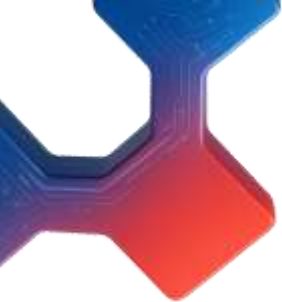
- Observe que o vetor transformado é 3D em vez de 2D. Agora, vamos ver o que acontece com dois vetores 2D, **a** e **b**, se aplicarmos esse mapeamento polinomial de segundo grau e, em seguida, calcularmos o produto escalar dos vetores transformados (veja a Equação abaixo).

$$\begin{aligned}\varphi(\mathbf{a})^T \varphi(\mathbf{b}) &= \begin{pmatrix} a_1^2 \\ \sqrt{2} a_1 a_2 \\ a_2^2 \end{pmatrix}^T \begin{pmatrix} b_1^2 \\ \sqrt{2} b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2 \\ &= (a_1 b_1 + a_2 b_2)^2 = \left( \begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = (\mathbf{a}^T \mathbf{b})^2\end{aligned}$$

E quanto a isso? O produto escalar dos vetores transformados é igual ao quadrado do produto escalar dos vetores originais:

$$\boldsymbol{\phi}(\mathbf{a})^T \boldsymbol{\phi}(\mathbf{b}) = (\mathbf{a}^T \mathbf{b})^2$$





## 4. SVMs kernalizados

- A função  $\mathbf{K}(\mathbf{a}, \mathbf{b}) = (\mathbf{a}^\top \mathbf{b})^2$  é um **kernel polinomial** de segundo grau. Em aprendizado de máquina, um **kernel** é uma função capaz de calcular o produto escalar  $\boldsymbol{\phi}(\mathbf{a})^\top \boldsymbol{\phi}(\mathbf{b})$  usando apenas os vetores originais  $\mathbf{a}$  e  $\mathbf{b}$ , sem precisar computar (ou mesmo conhecer) a transformação  $\boldsymbol{\phi}$ . Segue uma lista alguns dos **kernels** mais comumente usados.

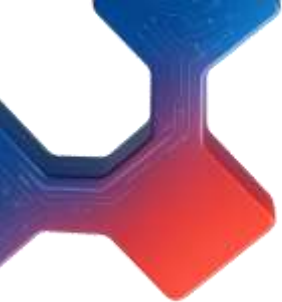
Linear:  $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$

Polynomial:  $K(\mathbf{a}, \mathbf{b}) = (\gamma \mathbf{a}^\top \mathbf{b} + r)^d$

Gaussian RBF:  $K(\mathbf{a}, \mathbf{b}) = \exp \left( -\gamma \| \mathbf{a} - \mathbf{b} \|^2 \right)$

Sigmoid:  $K(\mathbf{a}, \mathbf{b}) = \tanh (\gamma \mathbf{a}^\top \mathbf{b} + r)$





## 4. SVMs kernalizados

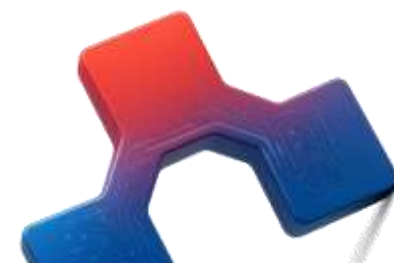
- Ainda há uma pontinha solta que devemos amarrar. A **Equação 5-4** mostra como passar da solução dual para a solução primal no caso de um classificador SVM linear. Mas se você aplicar o **kernel trick**, acaba com equações que incluem  $\phi(x^{(i)})$ .
- .De fato,  $w$  deve ter o mesmo número de dimensões que  $\phi(x^{(i)})$ , que pode ser enorme ou até infinito, então você não consegue computá-lo. Mas como fazer previsões sem conhecer  $w$  ? Bem, a boa notícia é que você pode substituir a fórmula de  $w$  da **Equação 5-4** na função de decisão para uma nova instância  $x^{(n)}$ , e você obtém uma equação com apenas produtos internos entre os vetores de entrada. Isso torna possível usar o **kernel trick** dada pela equação a seguir.

## 4. SVMs kernalizados

### Fazendo previsões com um SVM com kernel

$$\begin{aligned}h_{\widehat{\mathbf{w}}, \widehat{b}}(\varphi(\mathbf{x}^{(n)})) &= \widehat{\mathbf{w}}^\top \varphi(\mathbf{x}^{(n)}) + \widehat{b} = \left( \sum_{i=1}^m \widehat{\alpha}^{(i)} t^{(i)} \varphi(\mathbf{x}^{(i)}) \right)^\top \varphi(\mathbf{x}^{(n)}) + \widehat{b} \\&= \sum_{i=1}^m \widehat{\alpha}^{(i)} t^{(i)} (\varphi(\mathbf{x}^{(i)})^\top \varphi(\mathbf{x}^{(n)})) + \widehat{b} \\&= \sum_{\substack{i=1 \\ \widehat{\alpha}^{(i)} > 0}}^m \widehat{\alpha}^{(i)} t^{(i)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(n)}) + \widehat{b}\end{aligned}$$

- Note que, como  $\alpha_i \neq 0$  apenas para os vetores de suporte, fazer previsões envolve calcular o produto escalar do novo vetor de entrada  $\mathbf{x}^{(n)}$  apenas com os vetores de suporte, e não com todas as instâncias de treinamento. Claro, é necessário usar o mesmo truque para calcular o termo de viés  $\mathbf{b}$  como veremos a seguir.

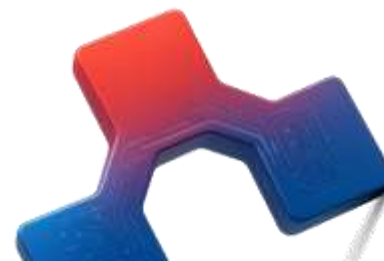


## 4. SVMs kernalizados

Usando o truque do kernel para calcular o termo de viés

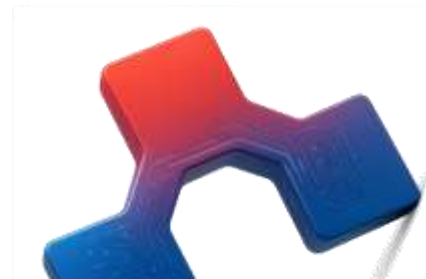
$$\begin{aligned}\hat{b} &= \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \left( t^{(i)} - \hat{\mathbf{w}}^\top \varphi(\mathbf{x}^{(i)}) \right) = \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \left( t^{(i)} - \left( \sum_{j=1}^m \hat{\alpha}^{(j)} t^{(j)} \varphi(\mathbf{x}^{(j)}) \right)^\top \varphi(\mathbf{x}^{(i)}) \right) \\ &= \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \left( t^{(i)} - \sum_{\substack{j=1 \\ \hat{\alpha}^{(j)} > 0}}^m \hat{\alpha}^{(j)} t^{(j)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \right)\end{aligned}$$

**Se você está começando a sentir dor de cabeça, isso é perfeitamente normal: é um efeito colateral infeliz do truque do kernel.**



## 5. Árvores de Decisão

- Árvores de decisão são algoritmos de aprendizado de máquina versáteis, capazes de realizar tarefas de classificação, regressão e até mesmo tarefas de saída múltipla. São algoritmos poderosos, capazes de ajustar conjuntos de dados complexos. Por exemplo, treinamos um modelo **DecisionTreeRegressor** no conjunto de dados de habitação da Califórnia.
- Árvores de decisão também são os componentes fundamentais das florestas aleatórias (**random forests**), que estão entre os algoritmos de aprendizado de máquina mais poderosos disponíveis atualmente.
- Nesta aula, começaremos discutindo como **treinar, visualizar e fazer previsões** com árvores de decisão. Em seguida, abordaremos o algoritmo de treinamento **CART** utilizado pelo **Scikit-Learn** e exploraremos como regularizar árvores e utilizá-las em tarefas de regressão. Por fim, discutiremos algumas das limitações das árvores de decisão.





## 5. Árvores de Decisão

- Para entender árvores de decisão, vamos construir uma e observar como ela faz previsões. O código a seguir treina um **DecisionTreeClassifier** no conjunto de dados *íris*.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris(as_frame=True)
X_iris = iris.data[["petal length (cm)", "petal width (cm)"]].values
y_iris = iris.target

tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf.fit(X_iris, y_iris)
```

Você pode visualizar a árvore de decisão treinada usando primeiro a função **export\_graphviz()** para gerar um arquivo de definição de gráfico chamado *iris\_tree.dot*:

```
from sklearn.tree import export_graphviz

export_graphviz(
    tree_clf,
    out_file="iris_tree.dot",
    feature_names=["petal length (cm)", "petal width (cm)"],
    class_names=iris.target_names,
    rounded=True,
    filled=True
)
```

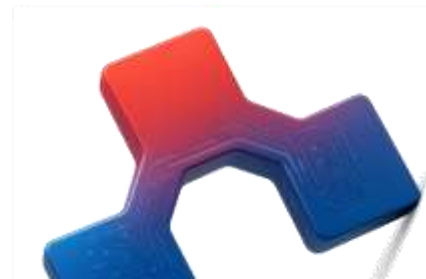


## 5. Árvores de Decisão

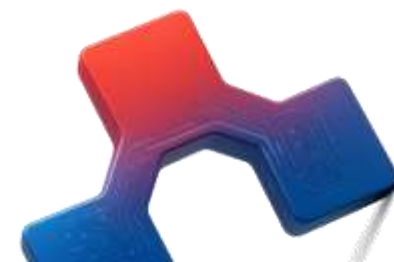
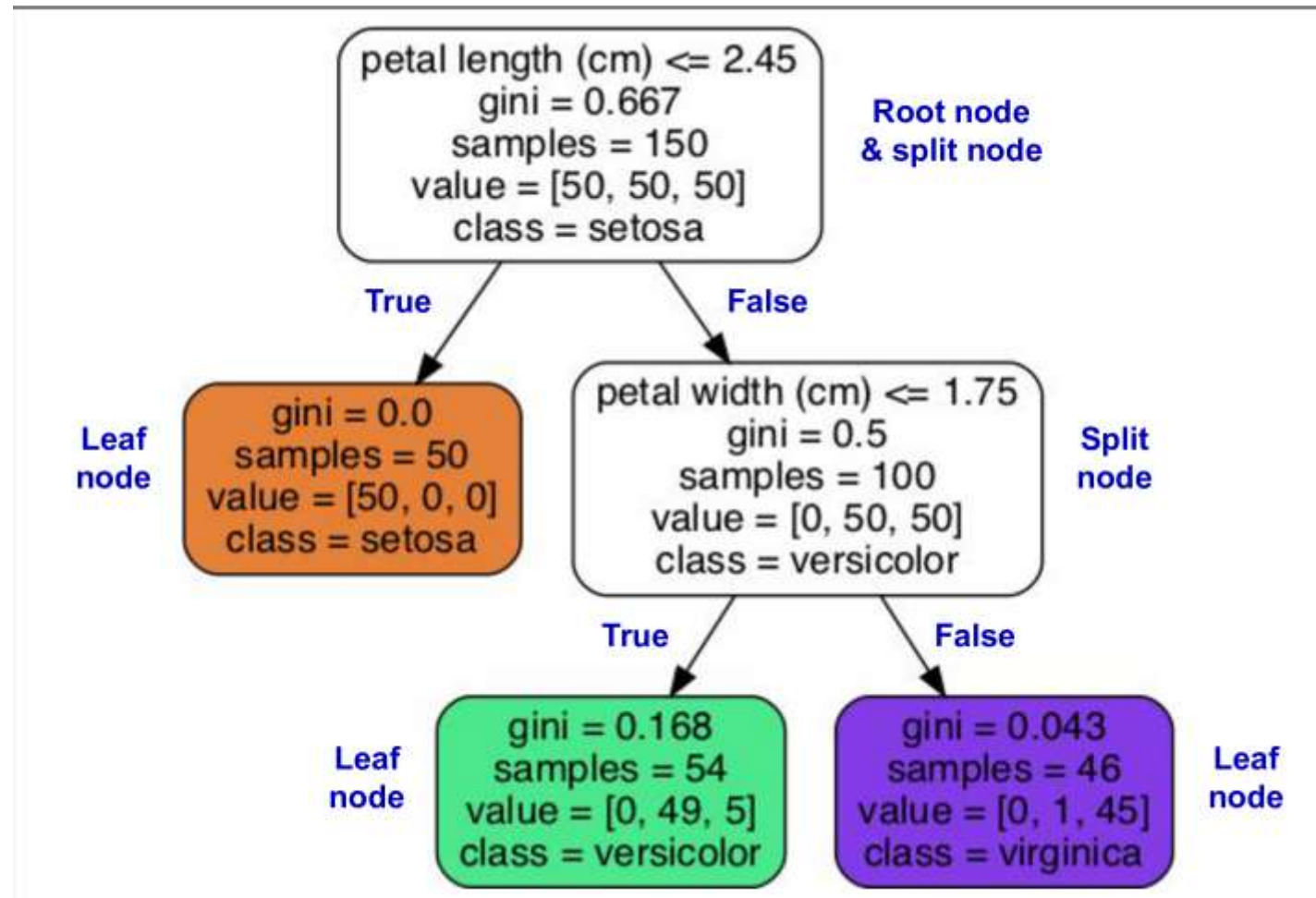
- Em seguida, você pode usar **graphviz.Source.from\_file()** para carregar e exibir o arquivo em um notebook Jupyter:

```
from graphviz import Source  
  
Source.from_file("iris_tree.dot")
```

- Sua primeira árvore de decisão será parecida com a figura a seguir:



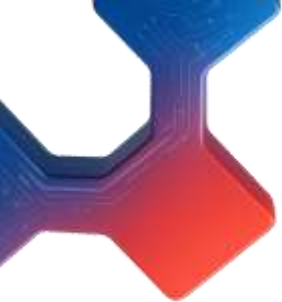
## 5. Árvores de Decisão



## 5. Fazendo Previsões

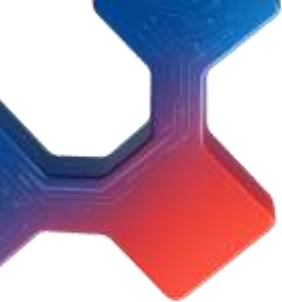
- Vamos ver como a árvore representada na Figura 6-1 faz previsões. Suponha que você encontre uma flor *iris* e queira classificá-la com base em suas pétalas. Você começa no nó raiz (profundidade 0, no topo): esse nó pergunta se o comprimento da pétala da flor é menor que 2,45 cm. Se for, você desce para o nó filho esquerdo da raiz (profundidade 1, à esquerda). Nesse caso, trata-se de um nó folha (ou seja, não possui nós filhos), então não faz mais perguntas: basta observar a classe prevista para esse nó, e a árvore de decisão prevê que sua flor é uma **Iris setosa** (classe = setosa).





## 5. Fazendo Previsões

- Agora, suponha que você encontre outra flor, e desta vez o comprimento da pétala seja maior que 2,45 cm. Você novamente começa no nó raiz, mas agora desce para o nó filho direito (profundidade 1, à direita). Este não é um nó folha, mas um nó de divisão, então ele faz outra pergunta: a largura da pétala é menor que 1,75 cm? Se for, sua flor provavelmente é uma **Iris versicolor** (profundidade 2, à esquerda). Caso contrário, é provavelmente uma **Iris virginica** (profundidade 2, à direita). É realmente simples assim.



## 5. Fazendo Previsões

- O atributo **sample** de um nó conta quantas instâncias de treinamento se aplicam a ele. Por exemplo, **100** instâncias de treinamento têm comprimento de pétala maior que **2,45** cm (profundidade 1, à direita), e dessas **100, 54** têm largura de pétala menor que 1,75 cm (profundidade 2, à esquerda).
- O atributo **value** de um nó indica quantas instâncias de treinamento de cada classe se aplicam a ele: por exemplo, o nó inferior direito se aplica a 0 **Iris setosa**, 1 **Iris versicolor** e 45 **Iris virginica**.
- O atributo **gini** de um nó mede sua impureza de Gini: um nó é “puro” ( $\text{gini} = 0$ ) se todas as instâncias de treinamento que se aplicam a ele pertencem à mesma classe

## 5. Fazendo Previsões

- Por exemplo, como o nó da profundidade **1** à esquerda se aplica apenas às instâncias de **Iris setosa**, ele é puro e sua impureza de Gini é **0**.
- A Equação a seguir mostra como o algoritmo de treinamento calcula a impureza de Gini  $G_i$  do  $i$ -ésimo nó. O nó da profundidade **2** à esquerda tem uma impureza de Gini igual a:

$$G_i = 1 - (0/54)^2 - (49/54)^2 - (5/54)^2 \approx 0,168$$

$$G_i = 1 - \sum_{k=1}^n p_{i,k}^2$$

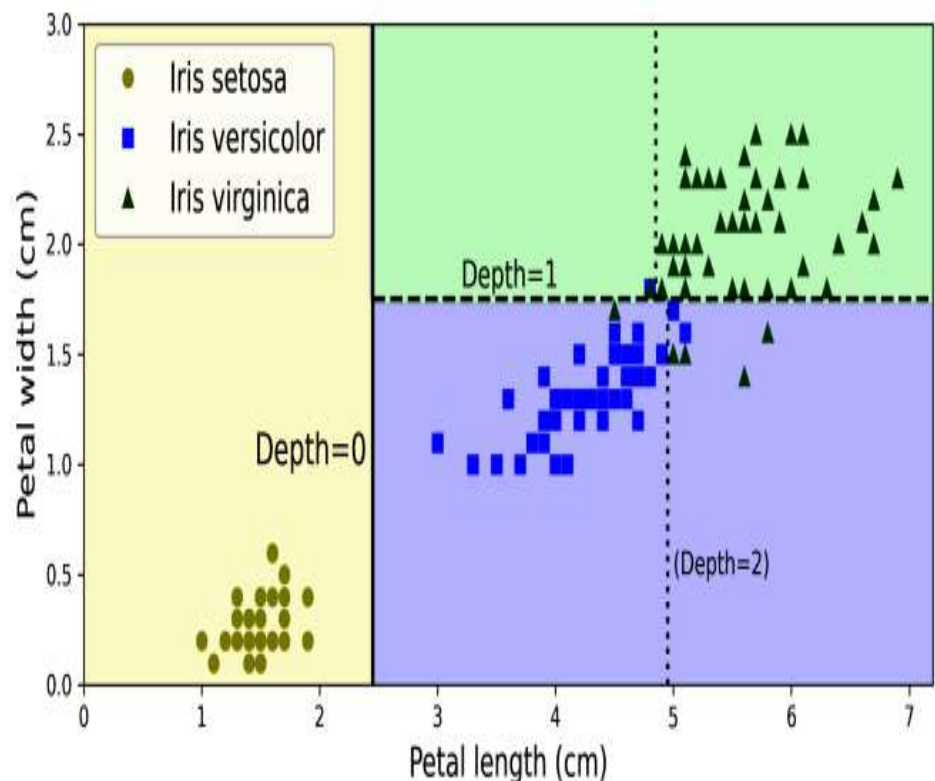
*Nesta equação:*

- $G_i$  é a impureza de Gini do  $i$ -ésimo nó.
- $p_{i,k}$  é a proporção de instâncias da classe  $k$  entre as instâncias de treinamento no  $i$ -ésimo nó.



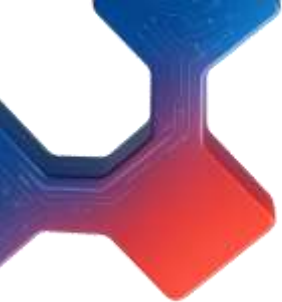


## 5. Fazendo Previsões



- Ao lado mostra os limites de decisão desta árvore de decisão. A linha vertical grossa representa o limite de decisão do nó raiz (profundidade 0): comprimento da pétala = 2,45 cm. Como a área à esquerda é pura (apenas **Iris setosa**), ela não pode ser dividida further.
- No entanto, a área à direita é impura, então o nó direito da profundidade 1 a divide na largura da pétala = 1,75 cm (representado pela linha tracejada). Como **max\_depth** foi definido como 2, a árvore de decisão para ali. Se você definir **max\_depth** como 3, então os dois nós da profundidade 2 adicionariam cada um outro limite de decisão (representado pelas duas linhas verticais pontilhadas).



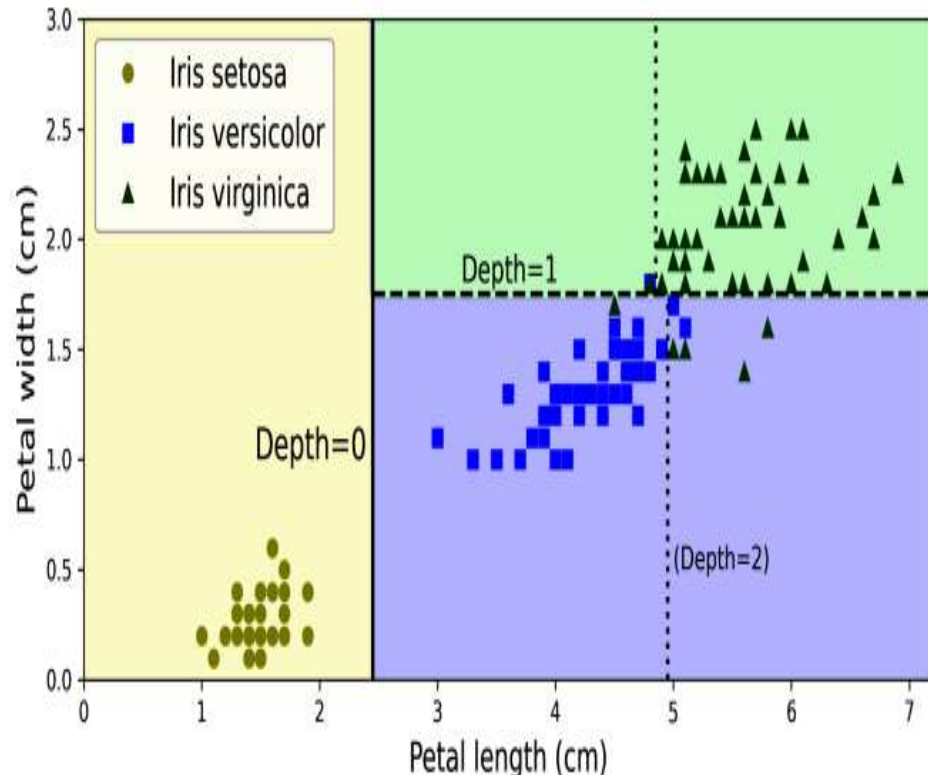


## 5. Estimando Probabilidades de Classe

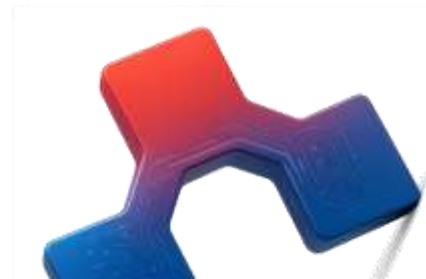
- Uma árvore de decisão também pode estimar a probabilidade de uma instância pertencer a uma determinada classe  $k$ . Primeiro, ela percorre a árvore para encontrar o nó folha correspondente a essa instância e, em seguida, retorna a proporção de instâncias de treinamento da classe  $k$  nesse nó.
- Por exemplo, suponha que você tenha encontrado uma flor cujas pétalas têm 5 cm de comprimento e 1,5 cm de largura. O nó folha correspondente é o nó da profundidade 2 à esquerda, então a árvore de decisão fornece as seguintes probabilidades: 0 % para **Iris setosa** (0/54), 90,7 % para **Iris versicolor** (49/54) e 9,3 % para **Iris virginica** (5/54).
- Se você solicitar que a árvore preveja a classe, ela retorna **Iris versicolor** (classe 1), pois é a que apresenta a maior probabilidade. Vamos verificar isso:

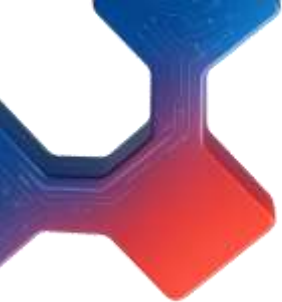
## 5. Estimando Probabilidades de Classe

```
>>> tree_clf.predict_proba([[5, 1.5]]).round(3)
array([[0.    , 0.907, 0.093]])
>>> tree_clf.predict([[5, 1.5]])
array([1])
```



- Perfeito! Observe que as probabilidades estimadas seriam idênticas em qualquer outro ponto do retângulo inferior direito da Figura ao lado — por exemplo, se as pétalas tivessem 6 cm de comprimento e 1,5 cm de largura (mesmo que pareça óbvio que, nesse caso, seria mais provável que fosse uma **Iris virginica**).

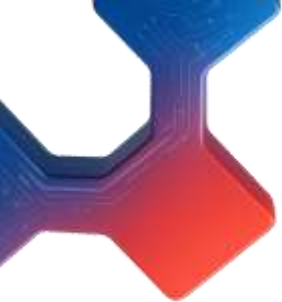




## 6. O Algoritmo de Treinamento CART

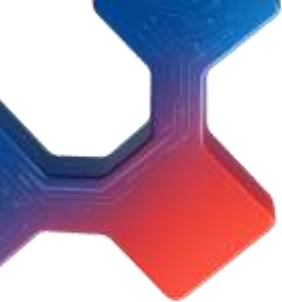
- O Scikit-Learn utiliza o algoritmo **Classification and Regression Tree (CART)** para treinar árvores de decisão (também chamado de “crescimento” de árvores). O algoritmo funciona dividindo inicialmente o conjunto de treinamento em dois subconjuntos usando uma única característica  $k$  e um limiar  $t_k$  (por exemplo, “comprimento da pétala  $\leq 2,45$  cm”).
- Como ele escolhe  $k$  e  $t_k$ ? O algoritmo procura pelo par  $(k, t_k)$  que produza os subconjuntos mais puros, ponderados pelo tamanho deles. A Equação a seguir apresenta a função de custo que o algoritmo tenta minimizar.

$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$



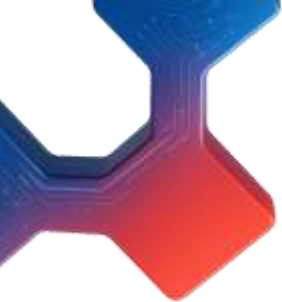
## 6. O Algoritmo de Treinamento CART

- Uma vez que o algoritmo **CART** tenha dividido com sucesso o conjunto de treinamento em dois, ele divide os subconjuntos usando a mesma lógica, depois os sub-subconjuntos e assim por diante, de forma recursiva.
- Ele para a recursão quando atinge a profundidade máxima (definida pelo hiperparâmetro **max\_depth**) ou se não conseguir encontrar uma divisão que reduza a impureza. Outros hiperparâmetros (descritos a seguir) controlam condições adicionais de parada: **min\_samples\_split**, **min\_samples\_leaf**, **min\_weight\_fraction\_leaf** e **max\_leaf\_nodes**.



## 7. Complexidade Computacional

- Fazer previsões requer percorrer a árvore de decisão desde a raiz até um nó folha. Árvores de decisão geralmente são aproximadamente balanceadas, então percorrer a árvore exige passar por aproximadamente  **$O(\log_2(m))$**  nós, onde  **$\log_2(m)$**  é o logaritmo binário de  **$m$** , igual a  **$\log(m) / \log(2)$** .
- Como cada nó exige apenas verificar o valor de uma característica, a complexidade geral de previsão é  **$O(\log_2(m))$** , independentemente do número de características. Portanto, as previsões são muito rápidas, mesmo ao lidar com conjuntos de treinamento grandes.
- O algoritmo de treinamento compara todas as características (ou menos, se **max\_features** estiver definido) em todas as amostras de cada nó. Comparar todas as características em todas as amostras de cada nó resulta em uma complexidade de treinamento de  **$O(n \times m \log_2(m))$** .



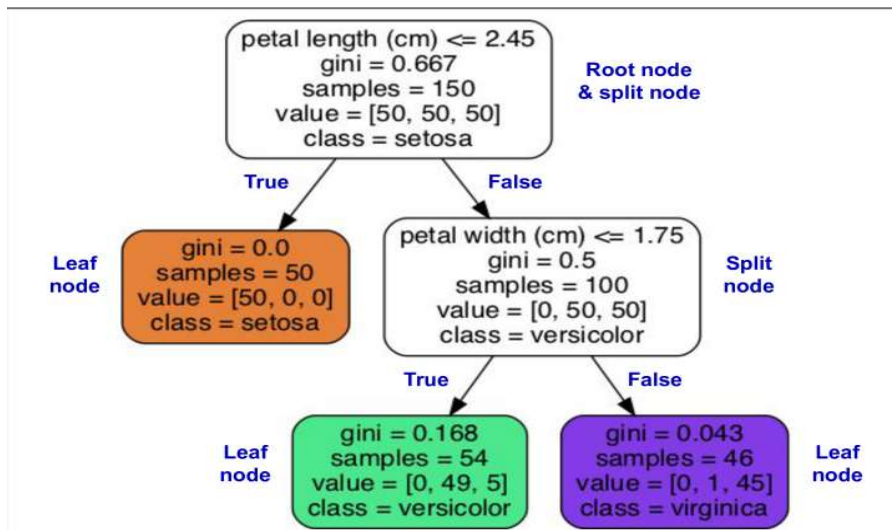
## 7. Impureza de Gini ou Entropia?

- Por padrão, a classe **DecisionTreeClassifier** utiliza a medida de impureza de Gini, mas você pode selecionar a medida de impureza de entropia definindo o hiperparâmetro **criterion** como "entropy".
- O conceito de entropia originou-se na termodinâmica como uma medida de desordem molecular: a entropia tende a 0 quando as moléculas estão estáticas e bem organizadas. Posteriormente, a entropia se espalhou para uma variedade de domínios, incluindo a teoria da informação de Shannon, onde mede o conteúdo médio de informação de uma mensagem. A **entropia é 0 quando todas as mensagens são idênticas**.

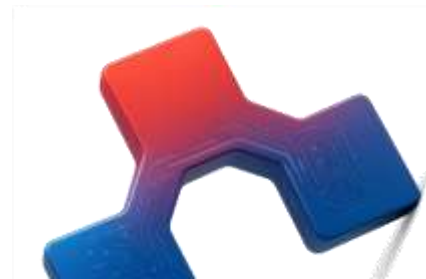


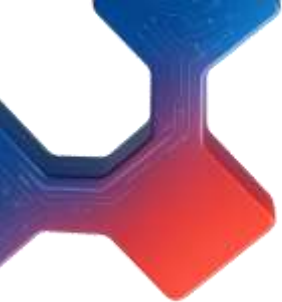
# 7. Impureza de Gini ou Entropia?

- No aprendizado de máquina, a entropia é frequentemente usada como medida de impureza: a entropia de um conjunto é 0 quando ele contém instâncias de apenas uma classe. A Equação a seguir mostra a definição da entropia do  $i$ -ésimo nó.
- Por exemplo, o nó da profundidade 2 à esquerda na figura abaixo tem entropia igual a:
  - $(49/54) \log_2 (49/54) - (5/54) \log_2 (5/54) \approx 0,445$



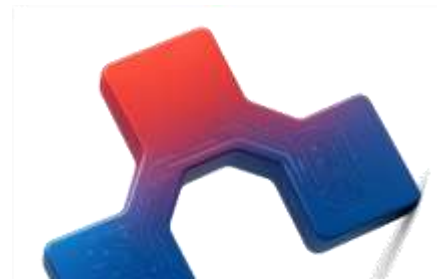
$$H_i = - \sum_{\substack{k=1 \\ p_{i,k} \neq 0}}^n p_{i,k} \log(p_{i,k})$$

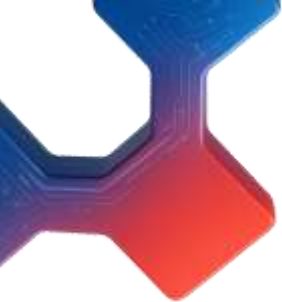




## 7. Impureza de Gini ou Entropia?

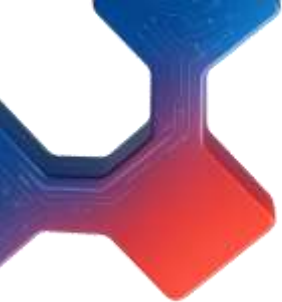
- Então, você deve usar impureza de Gini ou entropia? A verdade é que, na maioria das vezes, isso não faz muita diferença: ambos conduzem a árvores semelhantes. A impureza de Gini é ligeiramente mais rápida de calcular, sendo, portanto, uma boa escolha padrão. No entanto, quando há diferenças, a impureza de Gini tende a isolar a classe mais frequente em seu próprio ramo da árvore, enquanto a entropia tende a produzir árvores um pouco mais balanceadas.





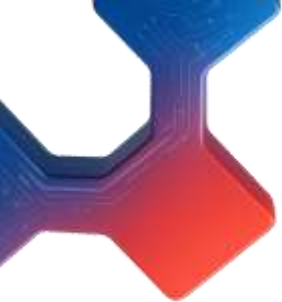
## 8. Hiperparâmetros de Regularização

- Árvores de decisão fazem pouquíssimas suposições sobre os dados de treinamento (ao contrário de modelos lineares, que assumem, por exemplo, que os dados são lineares). Se deixada sem restrições, a estrutura da árvore se adaptará aos dados de treinamento, ajustando-os muito de perto — na verdade, provavelmente ocorrendo **overfitting**. Tal modelo é frequentemente chamado de **modelo não paramétrico**, não porque não possua parâmetros (ele geralmente possui muitos), mas porque o número de parâmetros não é determinado antes do treinamento, de modo que a estrutura do modelo é livre para se ajustar aos dados. Em contraste, um **modelo paramétrico**, como um modelo linear, possui um número predeterminado de parâmetros, de forma que seu grau de liberdade é limitado, reduzindo o risco de overfitting (mas aumentando o risco de **underfitting**).



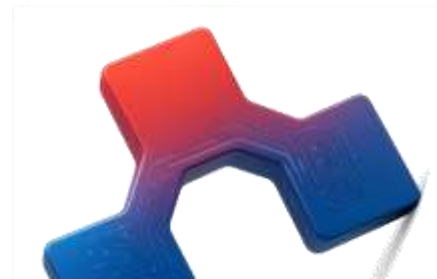
## 8. Hiperparâmetros de Regularização

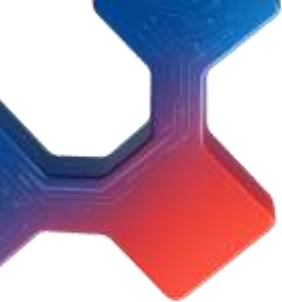
- Para evitar o **overfitting** nos dados de treinamento, é necessário restringir a liberdade da árvore de decisão durante o treinamento. Como você já sabe, isso é chamado de **regularização**.
- Os hiperparâmetros de regularização dependem do algoritmo utilizado, mas, geralmente, é possível ao menos restringir a **profundidade máxima** da árvore de decisão. No Scikit-Learn, isso é controlado pelo hiperparâmetro **max\_depth**. O valor padrão é **None**, o que significa profundidade ilimitada.
- Reduzir **max\_depth** regulariza o modelo e, assim, diminui o risco de overfitting.



## 8. Hiperparâmetros de Regularização

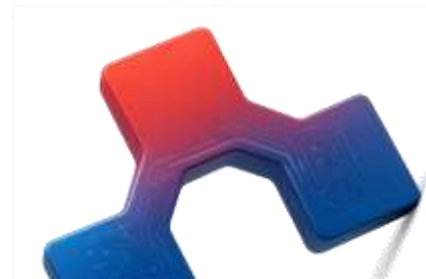
- A classe **DecisionTreeClassifier** possui alguns outros parâmetros que restringem de forma semelhante a forma da árvore de decisão:
- **max\_features**: Número máximo de características avaliadas para divisão em cada nó.
- **max\_leaf\_nodes**: Número máximo de nós folha.
- **min\_samples\_split**: Número mínimo de amostras que um nó deve ter antes de poder ser dividido.



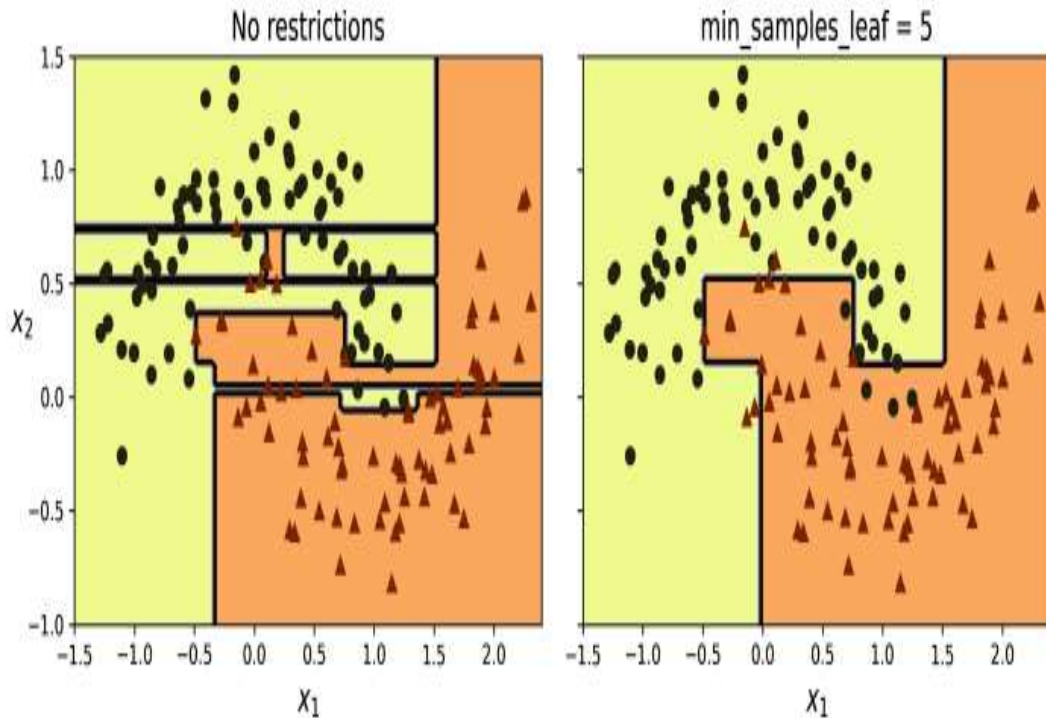


## 8. Hiperparâmetros de Regularização

- **min\_samples\_leaf**: Número mínimo de amostras que um nó folha deve ter para ser criado.
- **min\_weight\_fraction\_leaf**: Igual a **min\_samples\_leaf**, mas expresso como uma fração do número total de instâncias ponderadas.
- Aumentar os hiperparâmetros **min\_\*** ou reduzir os hiperparâmetros **max\_\*** regulariza o modelo.
- Vamos testar a regularização no conjunto de dados **moons**, apresentado no Capítulo 5. Treinaremos uma árvore de decisão sem regularização e outra com **min\_samples\_leaf = 5**.
- Aqui está o código; a Figura seguir mostra os limites de decisão de cada árvore:



## 8. Hiperparâmetros de Regularização



```
from sklearn.datasets import make_moons
```

```
X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)
```

```
tree_clf1 = DecisionTreeClassifier(random_state=42)
```

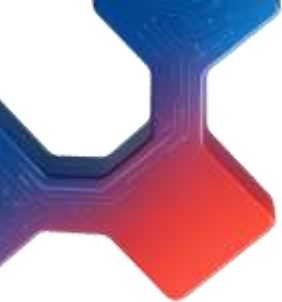
```
tree_clf2 = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
```

```
tree_clf1.fit(X_moons, y_moons)
```

```
tree_clf2.fit(X_moons, y_moons)
```

O modelo sem regularização à esquerda está claramente sofrendo **overfitting**, enquanto o modelo regularizado à direita provavelmente terá melhor capacidade de **generalização**. Podemos verificar isso avaliando ambas as árvores em um conjunto de teste gerado com uma semente aleatória diferente:





## 8. Hiperparâmetros de Regularização

- O modelo sem regularização à esquerda está claramente sofrendo **overfitting**, enquanto o modelo regularizado à direita provavelmente terá melhor capacidade de **generalização**. Podemos verificar isso avaliando ambas as árvores em um conjunto de teste gerado com uma semente aleatória diferente:

```
from sklearn.datasets import make_moons

X_moons, y_moons = make_moons(n_samples=150, noise=0.2, random_state=42)

tree_clf1 = DecisionTreeClassifier(random_state=42)
tree_clf2 = DecisionTreeClassifier(min_samples_leaf=5, random_state=42)
tree_clf1.fit(X_moons, y_moons)
tree_clf2.fit(X_moons, y_moons)
```

De fato, a segunda árvore apresenta uma **maior acurácia** no conjunto de teste.

## 9. Regressão

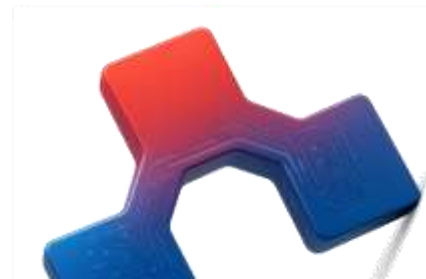
- Árvores de decisão também são capazes de realizar tarefas de **regressão**. Vamos construir uma **árvore de regressão** usando a classe **DecisionTreeRegressor** do Scikit-Learn, treinando-a em um conjunto de dados quadrático com ruído, definindo **max\_depth = 2**:

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

np.random.seed(42)
X_quad = np.random.rand(200, 1) - 0.5 # a single random input feature
y_quad = X_quad ** 2 + 0.025 * np.random.randn(200, 1)

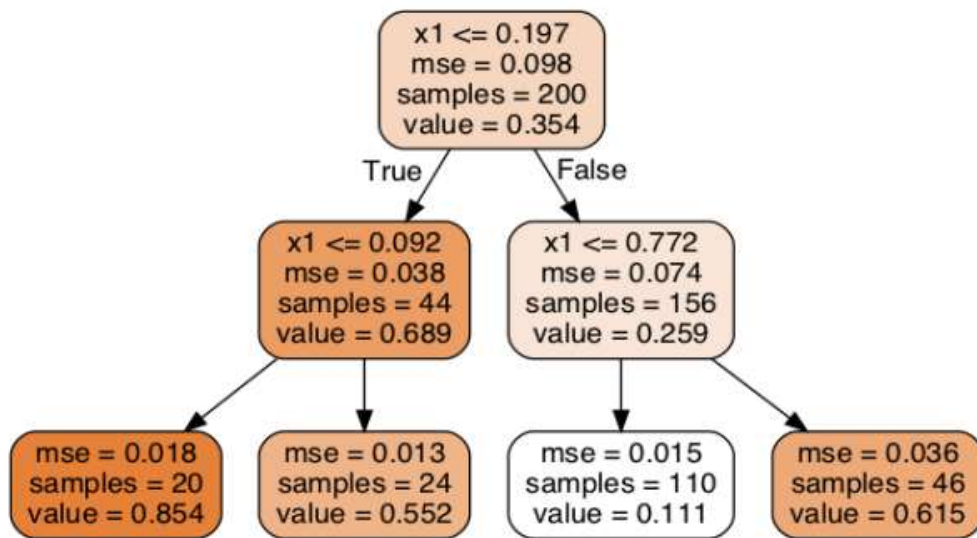
tree_reg = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg.fit(X_quad, y_quad)
```

- A árvore resultante está representada na figura a seguir:

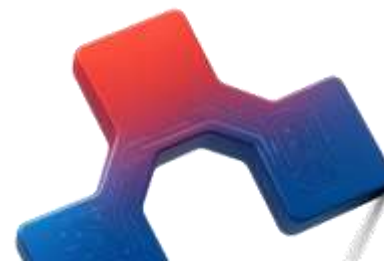


## 9. Regressão

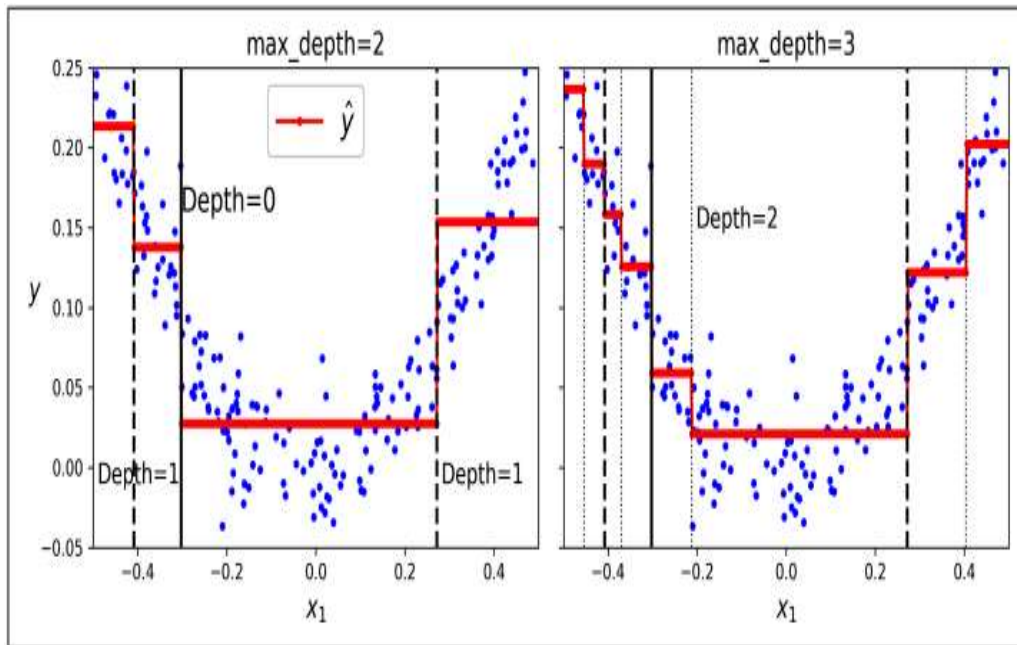
- Esta árvore se parece muito com a árvore de classificação que você construiu anteriormente. A principal diferença é que, em vez de prever uma classe em cada nó, ela prevê um valor. Por exemplo, suponha que você queira fazer uma previsão para uma nova instância com  $x_1 = 0.2$ .



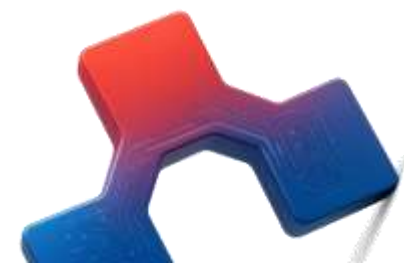
- O nó raiz pergunta se  $x_1 \leq 0.197$ . Como não é, o algoritmo segue para o nó filho à direita, que pergunta se  $x_1 \leq 0.772$ . Como é, o algoritmo vai para o nó filho à esquerda. Este é um nó folha, e ele prevê valor = 0.111. Essa previsão é a média dos valores alvo das 110 instâncias de treinamento associadas a este nó folha, resultando em um erro quadrático médio igual a 0.015 para essas 110 instâncias.



## 9. Regressão



As previsões deste modelo são representadas à esquerda na Figura ao lado. Se você definir `max_depth=3`, obterá as previsões representadas à direita. Observe como o valor previsto para cada região é sempre a média dos valores alvo das instâncias naquela região. O algoritmo divide cada região de uma forma que faz com que a maioria das instâncias de treinamento fique o mais próxima possível desse valor previsto.



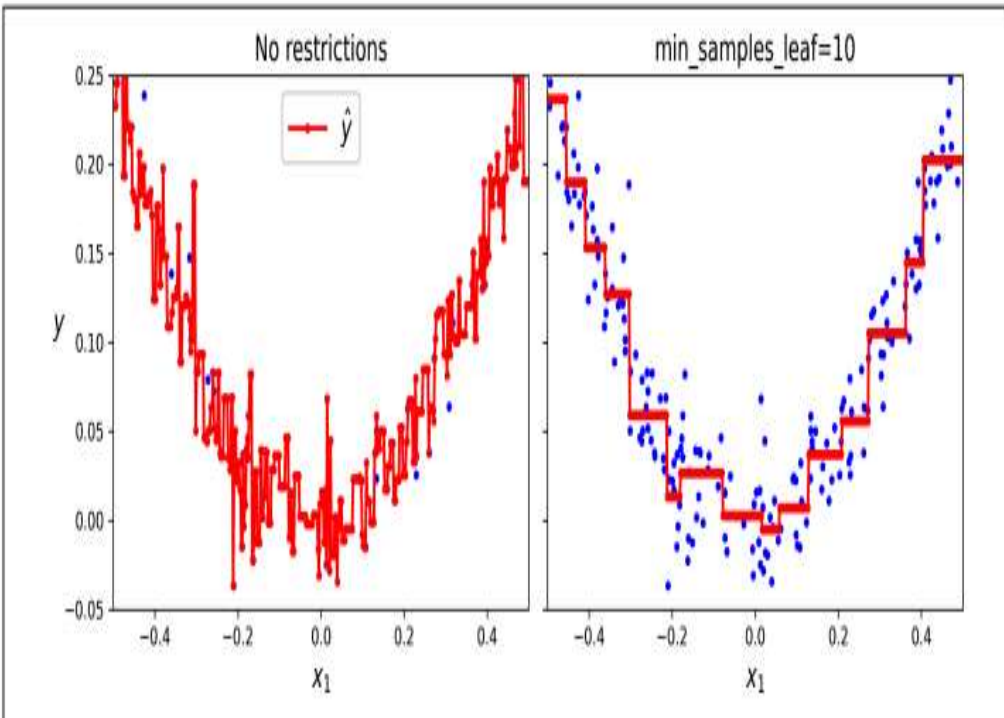
## 9. Regressão

- O algoritmo CART funciona como descrito anteriormente, exceto que, em vez de tentar dividir o conjunto de treinamento de forma a minimizar a impureza, ele agora tenta dividir o conjunto de treinamento de forma a minimizar o erro quadrático médio (MSE). A Equação a seguir mostra a função de custo que o algoritmo tenta minimizar.

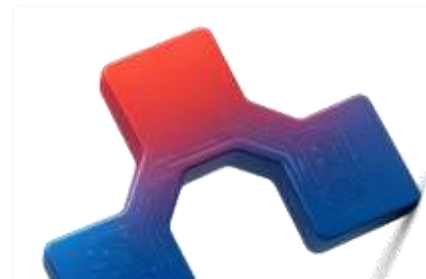
$$J(k, t_k) = \frac{m_{\text{left}}}{m} \text{MSE}_{\text{left}} + \frac{m_{\text{right}}}{m} \text{MSE}_{\text{right}} \quad \text{where} \quad \begin{cases} \text{MSE}_{\text{node}} = \frac{\sum_{i \in \text{node}} (\hat{y}_{\text{node}} - y^{(i)})^2}{m_{\text{node}}} \\ \hat{y}_{\text{node}} = \frac{\sum_{i \in \text{node}} y^{(i)}}{m_{\text{node}}} \end{cases}$$



## 9. Regressão

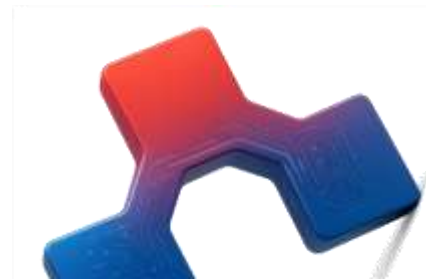
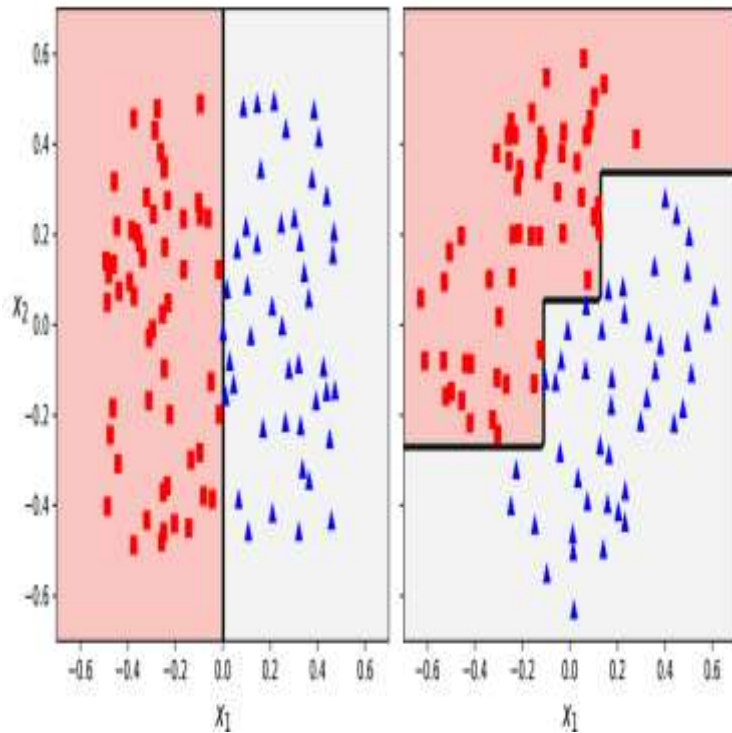


- Assim como em tarefas de classificação, árvores de decisão estão sujeitas a **overfitting** ao lidar com tarefas de regressão. Sem qualquer regularização (ou seja, usando os hiperparâmetros padrão), obtemos as previsões à esquerda na Figura ao lado. Essas previsões estão claramente ajustando excessivamente o conjunto de treinamento. Apenas definir **min\_samples\_leaf=10** resulta em um modelo muito mais razoável, representado à direita na Figura ao lado.



## 10. Sensibilidade à Orientação dos Eixos

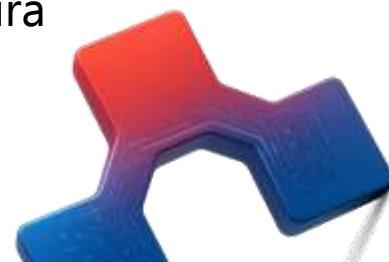
- Esperançosamente, até agora você já está convencido de que árvores de decisão têm muitas vantagens: elas são relativamente fáceis de entender e interpretar, simples de usar, versáteis e poderosas. No entanto, elas apresentam algumas limitações. Primeiro, como você deve ter percebido, **árvores de decisão preferem fronteiras de decisão ortogonais** (todas as divisões são perpendiculares a um eixo), o que as torna sensíveis à orientação dos dados. Por exemplo, a Figura ao lado mostra um conjunto de dados **linearmente separável** simples: à esquerda, uma árvore de decisão consegue dividi-lo facilmente, enquanto à direita, após o conjunto de dados ser rotacionado em  $45^\circ$ , a fronteira de decisão parece desnecessariamente convoluta. Embora ambas as árvores de decisão ajustem perfeitamente o conjunto de treinamento, é muito provável que o modelo à direita não generalize bem.





## 10. Sensibilidade à Orientação dos Eixos

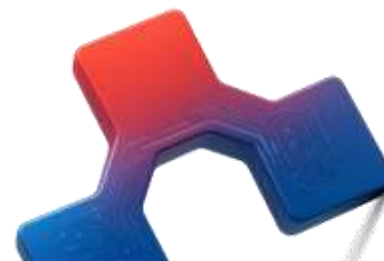
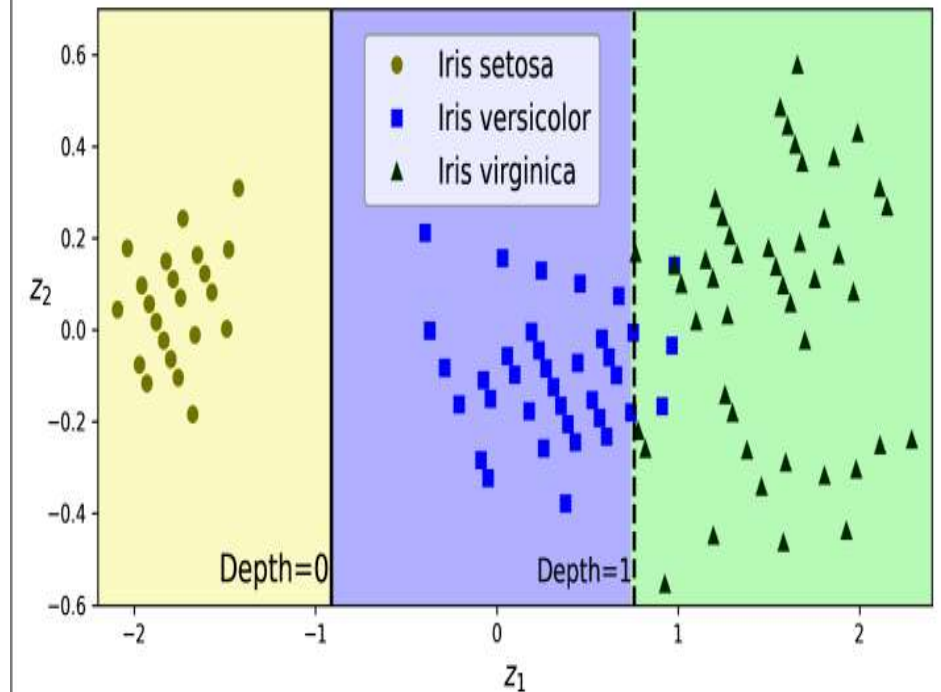
- Uma forma de limitar esse problema é escalonar os dados e, em seguida, aplicar uma transformação de análise de componentes principais (PCA). Veremos o PCA em detalhe mais na frente nesse curso, mas, por enquanto, você só precisa saber que ele rotaciona os dados de uma forma que reduz a correlação entre as variáveis, o que frequentemente (mas nem sempre) facilita o trabalho das árvores.
- Vamos criar um pequeno pipeline que escalona os dados e os rotaciona usando PCA, e então treinar um `DecisionTreeClassifier` com esses dados. A Figura a seguir mostra as fronteiras de decisão dessa árvore: como você pode ver, a rotação possibilita ajustar bem o conjunto de dados usando apenas uma variável,  $z_1$ , que é uma função linear do comprimento e da largura da pétala originais.



## 10. Sensibilidade à Orientação dos Eixos

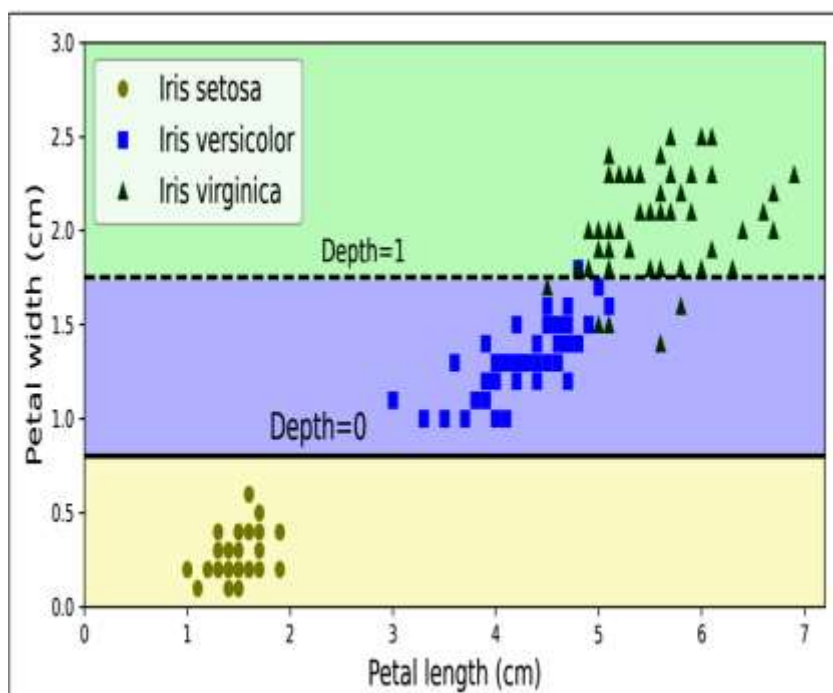
```
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

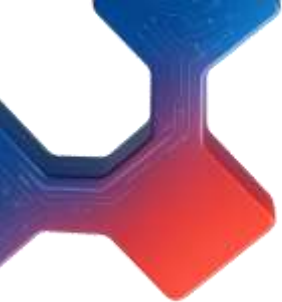
pca_pipeline = make_pipeline(StandardScaler(), PCA())
X_iris_rotated = pca_pipeline.fit_transform(X_iris)
tree_clf_pca = DecisionTreeClassifier(max_depth=2, random_state=42)
tree_clf_pca.fit(X_iris_rotated, y_iris)
```



# 11. Árvores de Decisão Possuem Alta Variância

- De forma mais geral, o principal problema das árvores de decisão é que elas possuem uma variância bastante alta: pequenas alterações nos hiperparâmetros ou nos dados podem gerar modelos muito diferentes. Na verdade, como o algoritmo de treinamento usado pelo **Scikit-Learn** é estocástico — ele seleciona aleatoriamente o conjunto de variáveis a ser avaliado em cada nó —, mesmo treinar a mesma árvore de decisão nos mesmos dados exatos pode produzir um modelo muito diferente, como o representado na Figura ao lado (a menos que você defina o hiperparâmetro **random\_state**).





# 11. Árvores de Decisão Possuem Alta Variância

- Felizmente, ao se fazer a média das previsões de várias árvores, é possível reduzir a variância de forma significativa. Um conjunto de árvores assim é chamado de **random forest**, e é um dos tipos de modelos mais poderosos disponíveis atualmente, como você verá na próxima aula.

